



University of Piraeus

School of Information and Communication Technologies

Department of Digital Systems

Level: Master's Programme of Studies

Advanced Topics in Cybersecurity and Artificial Intelligence

Hybrid Intrusion Detection System — ML + Snort

Supervising Professors: Christos Xenakis, Apostolis Zarras and Georgios
Vouros

Ioannis Kalaitzidis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

Introduction

Every organisational network faces daily breach attempts, from automated scans to targeted attacks. Effective detection requires two complementary approaches: signature-based rules that reliably catch known attacks, and anomaly/ML detection that identifies novel threats invisible to static rules.

The subject of this assignment is not merely to demonstrate that such a system detects attacks, but to systematically measure how much better the hybrid combination performs compared to each approach alone — and to be honest about the failures.

There are two major families of detection techniques, and the entire assignment rests on their difference. The first is signature-based detection (Snort): a list of rules that define exactly what an attack looks like.

The second family is anomaly or machine-learning based detection. Instead of explicit rules, the system learns from past normal traffic and flags anything that deviates significantly from that learned baseline.

The contrast between the two approaches can be illustrated with an analogy of two guards at an entrance. The first recognises only specific known faces — fast and certain for those faces, blind to anyone else. The second observes general behaviour patterns and flags anyone acting strangely — catches unknowns but may trigger on legitimate unusual behaviour.

Special emphasis was placed on the honesty of the evaluation, because of a trap that, if not considered, leads to impressive but misleading results: temporal leakage — using future data to train and then evaluating on the past.

- **Explainability:** the system explains why it made each decision, showing which traffic features mattered most and providing a human-readable summary.
- **Accountability:** every alert is accompanied by its source — which component (rule or model) triggered it — so an analyst can verify and act.
- **Honest reporting:** we did not hide the system's failures (such as model false positives) or embellish the results.

As the application domain we chose a web server inside an internal network, receiving attacks from an external attacker, with both malicious and legitimate traffic flowing simultaneously.

The assignment answers seven tasks: we will design the testbed and describe the adversary (Task 1), generate and label traffic (Task 2), write Snort rules (Task 3), train ML models (Task 4), implement hybrid fusion (Task 5), evaluate and provide explainability (Task 6), and examine evasion techniques (Task 7).

Task 1: Testbed Design and Threat Model

Before detecting attacks, one needs a safe place to execute and study them. This is called a testbed — a miniaturised, isolated laboratory that faithfully reproduces the relevant aspects of a real network.

Topology and target service

We set up two separate networks instead of one. The internal network is called HOME_NET (range 10.0.0.0/24, CIDR) and represents the protected zone containing the web server. The external network is EXTERNAL_NET (range 192.168.1.0/24, CIDR) and represents the internet from which the attacker operates.

The reason for choosing two networks rather than one is fundamental. If everything ran on the same machine (localhost), then the concept of 'traffic coming from outside' would not exist, and many network-level attacks would behave differently or not at all.

Threat Model: Who we are facing

To harden a system, one must first describe the adversary. Our attacker can send any traffic to HOME_NET, can observe the responses (open ports, service banners, timing), and possesses typical attacker tools (nmap, hping3, sqlmap, curl).

How we know what was an attack (ground truth)

To say whether the system answered correctly, we need the objective truth — which moments were actually attacks. This is called ground truth and is established entirely from the execution timestamps of the attack scripts, completely independently of any detection feature.

Why we kept the truth separate from the features the model sees

Here lies the most important point of our methodology. The truth (whether a window was an attack) is determined exclusively from the timestamps of the external attack scripts and is never derived from any traffic feature used by the detector.

The reason is as follows. If the characterisation of a window as an attack derived from a rule of the form 'if packets exceed threshold X, it is an attack', and the model also used that same feature for detection, then training and evaluation would be circular — the model would be learning to predict its own input criterion.

How the system is built

The system is not a single large program but a series of small Python programs, each with one job, called sequentially via a shell script.

Why results come out the same every time

Traffic generation and model training use a fixed random seed (1337). The seed guarantees that every run produces exactly the same synthetic traffic, the same data split, and therefore the same trained model — making every numerical result fully reproducible.

Log files (logs/)

The logs/ folder holds the system's evidence. Table 1 explains what each file contains and its purpose.

Αρχείο	Στήλες	Ρόλος
event_log.csv	attack_type, start_time, end_time, description	Πότε έτρεξε κάθε επίθεση. Πάνω σε αυτό στηρίζεται η επισήμανση των παραθύρων. Το διαβάζει το feature_extraction.py .
snort_alerts.csv	rel_time, sid, msg, src	Οι συναγερμοί του Snort, ένας ανά πακέτο. Αποδεικνύει ότι κάθε κανόνας πυροδοτείται.
snort_windows.csv	window_id, snort_detection, sids_fired	Το σήμα του Snort ανά παράθυρο. Είναι η γέφυρα προς το μοντέλο. Το hybrid_fusion.py τα ενώνει μέσω

Αρχείο	Στήλες	Ρόλος
		του window_id.

Files implementing Task 1: the documentation file TESTBED_DESIGN.md, which describes the laboratory topology and the threat model.

Task 2: Traffic Generation and Labelling

A detection system cannot be evaluated without data. We generated a realistic traffic stream of approximately 1.5 hours, combining both legitimate requests and six types of attacks.

Legitimate traffic

Legitimate traffic simulates the daily life of a server. It includes normal web page visits from the internal client, health-check requests, and regular API calls.

The attacks

We implemented six types of attacks, covering the required four malicious scenarios plus one low-and-slow attack and one stealth variant.

- **SYN scan:** the attacker sends connection requests to a very large number of ports — like knocking on every door to see which ones are open.
- **ICMP flood:** floods the server with high-frequency pings, trying to overload it.
- **Connection flood:** opens a very large number of full connections very quickly, exhausting server resources.
- **HTTP probe:** requests suspicious admin-like URLs (e.g. /phpmyadmin/) and uses the user-agent string of a known exploitation tool (sqlmap).
- **Stealth web probe:** makes the same suspicious requests but sparsely and one at a time, so the volume looks normal and it bypasses volume-based thresholds.
- **Low-and-slow SQLi:** sends an SQL injection attack split into small pieces with delays between them, staying below detection thresholds.

From packets to windows and class imbalance

We do not examine each packet individually but group packets into one-second windows and describe each with a set of statistical features (count, mean, standard deviation of inter-arrival times, unique sources, etc.).

Files	Τύπος επίθεσης	Επεισόδια	Attack windows
	SYN_SCAN	4	8
	ICMP_FLOOD	3	12
	CONN_FLOOD	3	8
	HTTP_ATTACK	4	19
	STEALTH_WEB	2	38
	SLOW_ATTACK	2	89
	Σύνολο	18	174

implementing Task 2: `traffic_generation/build_dataset.py` generates the traffic, and `ml_detector/feature_extraction.py` converts packets to feature windows.

Task 3: Snort Rule Development

Snort is a widely deployed, open-source intrusion detection system that operates on signatures. It reads traffic packet by packet and applies user-defined rules in real time.

A rule has two main parts. The first is what it looks for in the content (e.g. a specific word in an HTTP request). The second is optional threshold conditions (e.g. more than X packets per second from the same source).

- **sid 1000001: catches a request to a suspicious admin page (/phpmyadmin/) based on content, without a threshold, so it is detected even as a single packet.**
- **sid 1000002: catches the sqlmap tool fingerprint in the request header.**

- **sid 1000003:** catches a barrage of pings coming from outside, with a threshold of 30 per second per sender.
- **sid 1000004:** catches port scanning, with a threshold of 40 connection requests in 2 seconds per sender.
- **sid 1000005:** catches connection flooding, with a threshold of 20 per second per sender.

To confirm they work, we ran the rules over the recorded traffic and counted the alerts. All five rules fired correctly on their respective attacks.

sid	Κανόνας	Alerts
1000001	WEB Suspicious URI (phpmyadmin)	46
1000002	WEB sqlmap User-Agent	95
1000003	ICMP Echo flood (external)	1415
1000004	TCP SYN scan / port sweep	111
1000005	TCP connection flood (burst)	171

Verification with real Snort

Beyond the simulator, the rules were also verified with real Snort 3.12.2 in a Kali Linux environment, running offline on the pcap file.

Running real Snort 3.12.2 on Kali Linux over the pcap: loading the five rules and full analysis summary (24,110 packets analysed).

```
(kali@kali)-[~/Downloads/MTE25012_Snort]
$ snort -c snort_kali.lua -R rules_kali.rules -r pcaps/lab_traffic.pcap -A alert_fast -l logs -k none
o*)~ Snort++ 3.12.2.0

Loading snort_kali.lua:
  http_inspect
  alert_fast
  output
  ips
  stream_tcp
  search_engine
  packets
  active
  alerts
  daq
  decode
  host_cache
  host_tracker
  hosts
  process
  network
  stream
  stream_icmp
  binder
Finished snort_kali.lua:
Loading rule args:
Loading rules_kali.rules:
Finished rules_kali.rules:
Finished rule args:

ips policies rule stats
  id loaded shared enabled file
  1 5 0 5 snort_kali.lua

rule counts
  total rules loaded: 5
  text rules: 5
  option chains: 5
  chain headers: 2

port rule counts
  tcp udp icmp ip
  any 0 1 0
  dst 2 0 0
  total 2 0 1

service rule counts
  http: 2 0
  http2: 2 0
  http3: 2 0
  total: 6 0

fast pattern groups
  to_server: 6

search engine (ac_bnfa)
  instances: 6
  patterns: 6
  pattern chars: 54
  num states: 54
  num match states: 6

memory scale: KB
  total memory: 8.22656
  pattern memory: 0.28125
  match list memory: 0.689375
  transition memory: 6.58594
  fast pattern only: 6

pcap DAQ configured to read-file.
Commencing packet processing
Retry queue interval is: 200 ms
+ [0] pcaps/lab_traffic.pcap
- [0] pcaps/lab_traffic.pcap

Packet Statistics
daq
  pcaps: 1
  received: 24110
  analyzed: 24110
  allow: 24110
  rx_bytes: 4214664

codec
  total: 24110 (100.000%)
  icmp4: 1654 ( 6.860%)
  ipv4: 24110 (100.000%)
  tcp: 22456 ( 93.140%)

Module Statistics
binder
  raw_packets: 4147
  new_flows: 2354
  no_match: 1815
  inspects: 6505

detection
  analyzed: 24110
  hard_evals: 10998
  alerts: 1607
  total_alerts: 1607
  logged: 1607

ips_actions
  alert: 1607

normalizer
  test_tcp_trim_win: 8668
  test_tcp_ips_data: 69

search_engine
  non_qualified_events: 9464
  qualified_events: 1534
```

count

Sample alerts (logs/alert_fast.txt) and alert per rule:

Πλήθος alerts ανά κανόνα (sid): ICMP flood

1412,

SYN scan 73, connection flood 122:

```
(kali@kali)-[~/Downloads/MTE25012_Snort]
$ for s in 1000001 1000002 1000003 1000004 1000005; do echo -n "$s: "; grep -c $s logs/alert_fast.txt; done
1000001: 0
1000002: 0
1000003: 1412
1000004: 73
1000005: 122

(kali@kali)-[~/Downloads/MTE25012_Snort]
$

max: 2354
created: 2354
released: 2354
timeouts: 18
instantiated: 2354
setups: 2354
syn_ack_trackers: 2354
segs_queued: 2400
segs_released: 2400
overlaps: 115
payload_fully_trimmed: 46
syn_acks: 2354
fins: 4598
max_segs: 3
max_bytes: 200
zero_win_probes: 6547

Summary Statistics
timing
  runtime: 00:00:00
  seconds: 0.120730
  pkts/sec: 133403
```


Files implementing Task 3: the rules/local.rules file contains the five Snort rules, and rules/snort.lua the configuration for real Snort.

Task 4: ML Features and Models

Τι είναι τα features

An ML model does not see raw packets — it sees numbers. A feature is a number that describes something about a one-second window.

How a model is trained and tested

We split the data into two parts: 70% for training and 30% for testing. The model learns only from the first part and is evaluated exclusively on the second — data it has never seen.

The three models

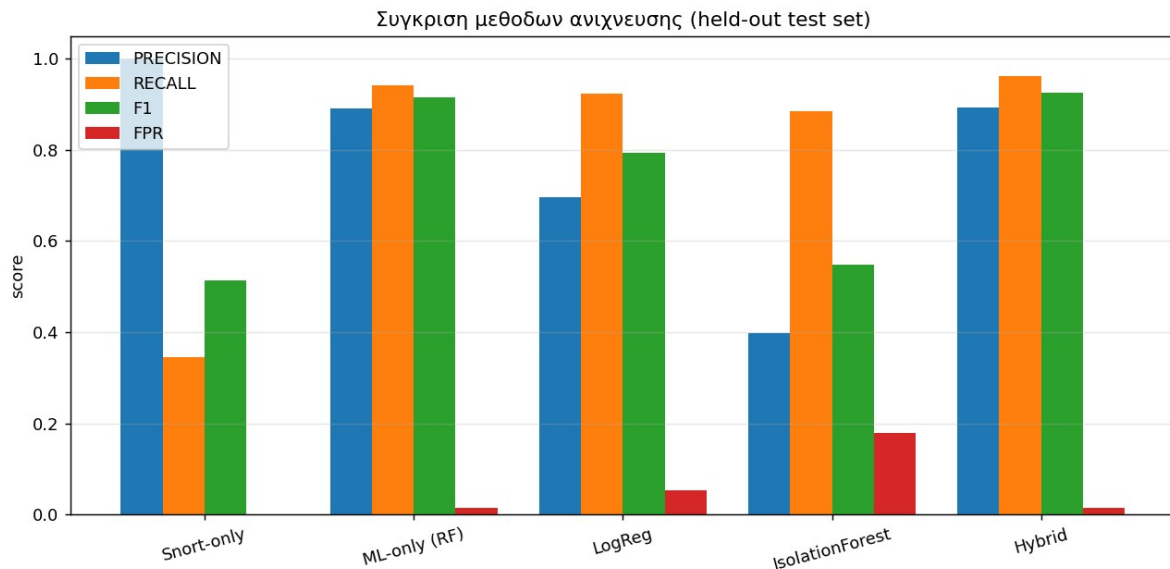
We tested three models, all from the course syllabus (Lectures 3 and 4). Random Forest is an ensemble of many small decision trees that vote on the final answer. Logistic Regression finds the best linear boundary between classes. Isolation Forest is an unsupervised model that isolates anomalies without needing attack labels during training.

Before reading the results, four concepts are needed. Precision answers: of all the system flagged as attacks, what fraction were real? Recall answers: of all real attacks, what fraction did the system catch? F1-score is the harmonic mean of the two. False Positive Rate (FPR) measures the fraction of legitimate traffic incorrectly flagged.

Μοντέλο	Precision	Recall	F1	FPR	AUC
Random Forest	0.891	0.942	0.916	0.015	0.991
Logistic Regression	0.696	0.923	0.793	0.053	0.968
Isolation Forest	0.397	0.885	0.548	0.178	0.908

Random Forest gives the best balance ($F1=0.916$). Logistic Regression catches almost all attacks but also flags too many legitimate windows (high FPR). Isolation Forest catches only a few (low Recall) but with very few false alarms.

We also examined which features mattered most in the Random Forest decisions. At the top were the timing features (standard deviation of inter-arrival times) and the connection count — exactly the features most distinctive between normal and attack traffic.



Files implementing Task 4: `ml_detector/feature_extraction.py` computes the features of each window, and `ml_detector/train_model.py` trains and evaluates the three models.

Task 5: Hybrid Fusion

Fusion means combining the opinions of the individual detectors into a single final decision. We assigned each window a combined risk score based on the three component signals.

The weights are not arbitrary. We gave the highest weight to Random Forest because it was the most reliable model, medium weight to Snort because its precision is perfect (0 FP) but its recall is low, and the lowest weight to Isolation Forest because it has high recall but also high FPR.

The final decision — whether to trigger an alert — is defined as the union of the two most precise detectors: we alert if either Snort or Random Forest triggers.

What happens when the two detectors disagree

We defined explicit rules for disagreements, because that is precisely where the value of a hybrid system is determined.

- **C1, when the signature speaks: if Snort fires but the model has a low score, we trust the rule (the signature is certain for known attacks).**
- **C2, when only the anomaly speaks: if neither Snort nor Random Forest fires but Isolation Forest shows a strong anomaly, we escalate for human review rather than blocking automatically.**
- **C3, when only the model speaks: if Random Forest fires while Snort is silent, we trust the model. This is precisely where the slow SQLi attack is caught.**

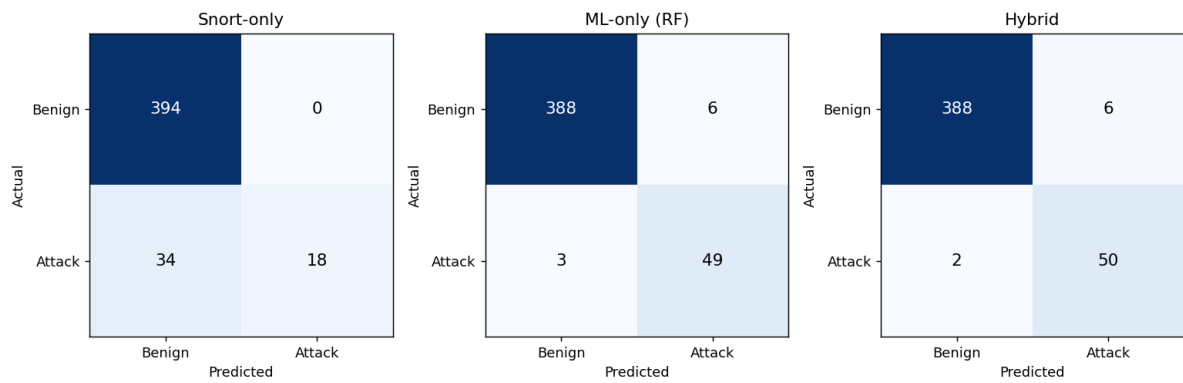
Files implementing Task 5: `ml_detector/hybrid_fusion.py` combines the Snort signal with the model scores and produces the final alert decision.

Task 6: Evaluation

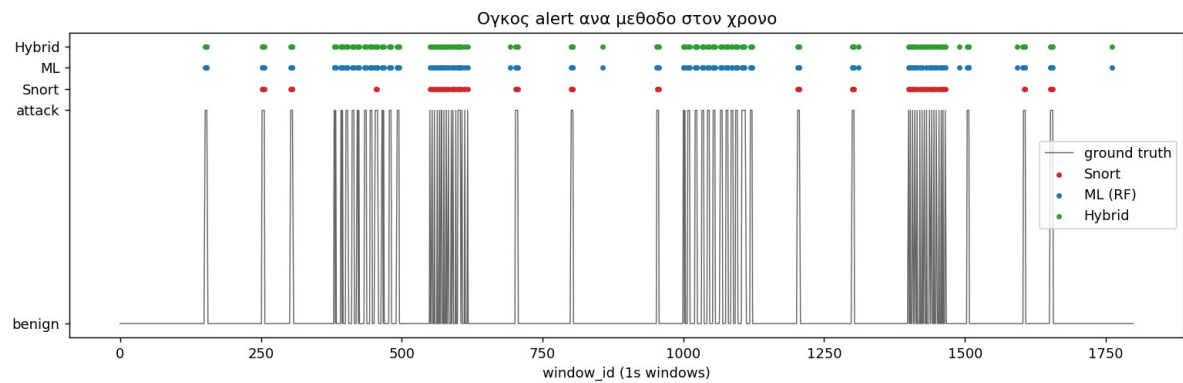
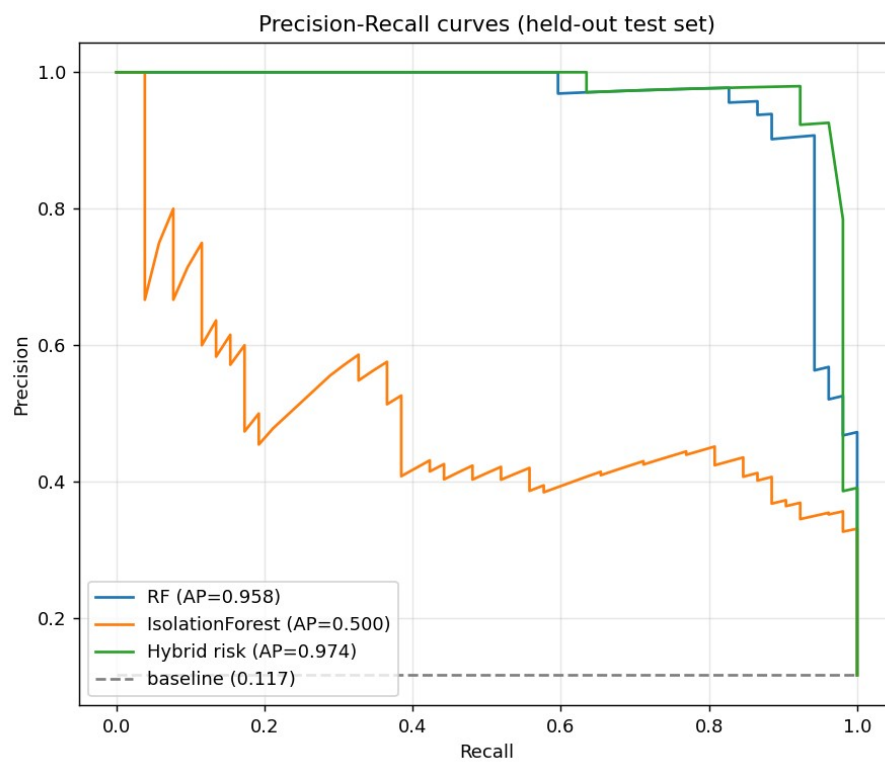
The following compares the three approaches in terms of performance. Table 5 compares all three on the test set.

Μέθοδος	TP	FP	FN	Precision	Recall	F1	FPR
Snort-only	18	0	34	1.000	0.346	0.514	0.000
ML-only (RF)	49	6	3	0.891	0.942	0.916	0.015
Hybrid	50	6	2	0.893	0.962	0.926	0.015

The columns TP, FP, FN are worth explaining, as they are the core of every evaluation (Lecture 5). TP (true positive) is an attack that the system correctly flagged. FP (false positive) is legitimate traffic incorrectly flagged. FN (false negative) is an attack that the system missed.



Since attacks are rare, we emphasised Precision-Recall curves instead of the more common ROC curves.



Three cases where the detectors disagree

The following cases come from real windows and clearly show why no single detector suffices.

- **The model catches, Snort misses:** in 98 windows the Random Forest catches attacks that Snort misses. These are mainly the low-and-slow SQLi attack spread over many packets with delays.
- **Snort catches, the model misses:** Snort catches stealthy requests that the model misses. In window 1428, the model score is low because the request volume is normal — but Snort catches the exact sqlmap user-agent string.
- **Model false positives:** in 7 windows Random Forest triggers incorrectly while Snort stays clean. These are legitimate windows with unusual traffic patterns that coincidentally resemble attack signatures.

How much to trust the numbers

The metrics should be read with awareness of sample size. With 52 attacks in the test set, each misclassification changes the metrics significantly.

Files implementing Task 6: `ml_detector/evaluation.py` computes the metrics and `evaluation.json` with detailed results per detector.

Task 7: Evasion Techniques and Critical Assessment

A smart attacker does not attack blindly — they try not to be detected. We examined three evasion techniques and which detector each one fools.

- **Content fragmentation (slow attack):** the signature is split across many packets so that none contains it in full. Fools primarily Snort; the model still sees anomalous timing.
- **Hiding inside low volume:** sparse, isolated requests so the volume appears normal. Fools primarily the model, which relies on bulk statistics; Snort still catches content-based patterns.

- **Staying below thresholds: maintaining the rate below rule limits. Fools rules that rely on thresholds; the model may still catch statistical deviations.**

The key conclusion is that every trick that neutralises one detector leaves traces visible to the other. That is precisely why the hybrid approach is robust.

However, we must be honest about the limits. The model was trained on known attack types. Against a genuinely novel attack (zero-day) that does not resemble the training data, it would fail.

Optional Task 1: Supervised vs Unsupervised Learning

This extension compares two fundamentally different ML approaches so as to highlight a key trade-off: performance vs label requirements.

The difference is reflected in the metrics. The two supervised models achieve clearly higher precision (0.891 for Random Forest, 0.867 for Logistic Regression) compared to Isolation Forest (0.471).

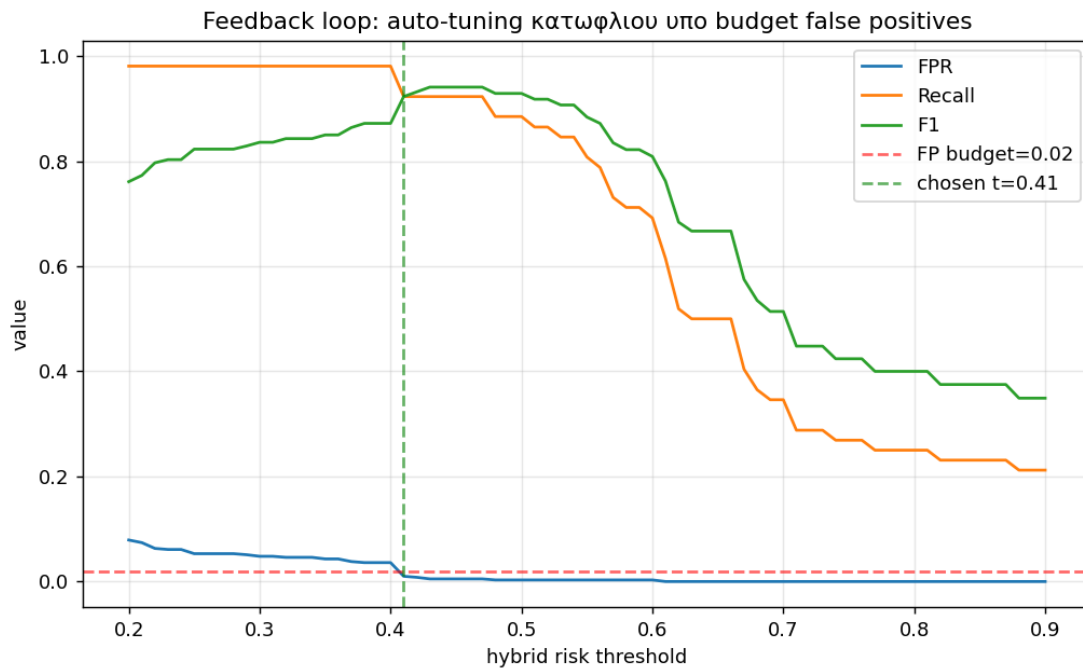
The finding has practical significance. In real security environments, confirmed attack labels are rare and expensive to obtain. The supervised models outperform when labels are available, but Isolation Forest is valuable for detecting anomalies in unlabelled traffic — a much more common real-world scenario.

Files implementing Optional Task 1: `ml_detector/train_model.py` trains all three models, and `ml_detector/evaluation.py` evaluates them.

Optional Task 2: Feedback Loop

The threshold above which the system triggers an alert is not a fixed parameter but a choice that depends on the relative cost of missed attacks versus false alarms.

The mechanism scans successive threshold values, measuring for each the false positive rate, recall and F1 score, and identifies the optimal operating point.



Files implementing Optional Task 2: the feedback mechanism is implemented in `ml_detector/evaluation.py`.

Execution conditions and parameters

So that anyone can reproduce exactly the same results, we record the execution conditions.

- **Data:** the analysis runs offline on a single traffic file (24,110 packets), without live capture.
- **Reproducibility:** fixed random seed (1337) in traffic generation and training, so every run produces identical results.
- **Models:** Random Forest (200 trees, max depth 8, class balancing), Logistic Regression, and Isolation Forest, from the scikit-learn library.

The entire chain — from traffic generation to evaluation — runs end-to-end via `run_all.sh` and is documented in `README.md`.

Limitations and validity threats

In the spirit of honest reporting (Lecture 7), the limits within which the conclusions hold are explicitly stated.

- **What is documented:** on the test set, Snort alone achieved precision 1.000 and recall 0.346, Random Forest F1=0.916, and the hybrid system F1=0.926.
- **Synthetic data:** results come from controlled, synthetically generated traffic. It is not documented that the performance translates to real-world heterogeneous networks.
- **Attack scope:** the metrics concern six specific attack types and do not generalise to unknown threats.
- **Parameter selection:** rule thresholds and fusion weights were chosen empirically and would need recalibration in a different environment.
- **Window size:** we work with one-second windows. Attacks that unfold over much shorter or longer timescales may not be captured correctly.

These limitations do not negate the central finding, but precisely delimit its scope of validity and indicate specific directions for future work.

Conclusion

We built a complete and reproducible framework to evaluate a hybrid intrusion detection system combining Snort signatures with three ML models.

Regarding the weakness of individual approaches: Snort showed perfect precision but low recall (0.346), missing 65% of attacks. Random Forest achieved good balance (F1=0.916) but produced false positives on 7 legitimate windows. Isolation Forest demonstrated the highest recall among the three but the lowest precision.

Regarding the solution: the hybrid system achieved the best F1 (0.926), outperforming every individual approach. The cases examined confirm the complementarity: Snort catches content-based threats invisible to the model, the model catches volume-based threats invisible to Snort.

A constant axis of the assignment was the independence of labelling from features, the reporting of failures, and the clear delimitation of validity — all principles of responsible AI examined in Lecture 7.

Deliverables and execution instructions

The submission is organised in a single folder with clear separation between code, data, and documentation. The code is fully documented and runs end-to-end via `run_all.sh`.

Task	Υλοποιείται από:
1. Testbed και μοντέλο απειλής	TESTBED_DESIGN.md, traffic_generation/build_dataset.py

Beyond the code, the submission includes this report, the documentation files, and the dependencies catalogue (requirements.txt).